Group Project 4

I. Software Architecture (80%)

1. Use your **existing GitHub repository** from the previous group project.
2. Update your local repository with the latest version of the main branch.
3. Create the following **User Stories** on your Kanban board:
    - **MVC-01: View Layer**
    - **MVC-02: Controller Layer**
4. **Assign each User Story** to a group member. Each member is responsible for updating the status of their assigned task.
5. Configure the Kanban board with at least four columns:
    - **To Do:** Tasks that have not started.
    - **In Progress:** Tasks currently being implemented.
    - **In Review:** Tasks submitted for testing or code review.
    - **Done:** Tasks completed and successfully merged into main.
6. Each member creates a **new Git branch** from the latest main branch for their assigned task.
7. After completing the implementation, each member pushes their branch to GitHub and creates a **Pull Request** for review.
8. The repository owner reviews each Pull Request and merges the changes into main after checking that they work correctly. Resolve any merge conflicts before merging.

II. Make sure that your board mirrors your current progress. (20%)

Appendix

1. Objective

Refactor the existing Java application into a simple Model–View–Controller (MVC) structure. Reuse the five classes already implemented.

2. Understand the Architecture

Layer	Responsibility	Classes	Acceptance check
Model	Hold data and business rules; enforce existing validation.	User, PropertyOwner, Property, Tenant, RentalContract	**No** Scanner, System.out.println menu, or View imports **inside** model classes.
View	Display menus, collect input, and show results/errors.	ConsoleView (new)	Menu and input-reading code **stay in** ConsoleView.
Controller	Receive View requests and coordinate model operations.	RentalController (new)	View calls controller methods for the required actions; controller delegates business rules to model.

Expected flow: User → ConsoleView → RentalController → Model → Controller → View. The View must not directly create rental contracts, change availability, or modify object fields.

3. Implementation instructions

Task A - Implement Controller

Create RentalController to expose at least these three operations: createProperty, rentProperty, and terminateContract. The Controller should call the existing model methods; it should not repeat Property or RentalContract validation.

Techniqual task:

1. Create the RentalController class inside the controller package.
2. Implement createProperty() to create a new property by calling the existing PropertyOwner method.
3. Implement rentProperty() to create a rental contract by calling the existing Tenant method.
4. Implement terminateContract() to terminate an existing rental contract by calling the appropriate RentalContract method

Example:

```
public Property createProperty(PropertyOwner owner, int id,
      String description, double rate) {
   return owner.createProperty(id, description, rate);
}
```

Task B - Implement View

Read inputs, call controller methods, and display the result or a useful error message. Existing object references may be stored in memory; persistence is not required.

Techniqual task:

1. Create ConsoleView with a simple text menu: (1) Create property, (2) Rent property, (3) Terminate contract, (4) Exit.

Example:

```
System.out.println("1. Create property  2. Rent  3. Terminate  4. Exit");
// Read choice; call controller; show success/error.
```

Task C - MVC Integration

- Main.java creates the Controller and View and launches the menu.

- Demonstrate a complete flow: create a property → rent it → terminate its contract. Confirm IsAvailable changes true → false → true.
- Demonstrate one invalid operation (e.g., rentalRate = 0 or endDate before startDate). The model must reject it; the View should display the error without crashing or corrupting stored state.